DIPLOMADO EN FULLSTACK DEVELOPER BACKEND FRONTEND

DESARROLLO BACKEND CON DJANGO

MÉTODO DE EVALUACIÓN

Asistencia	10%
Participación en clases	10%
Examen	40%
Proyecto	40%

TABLA DE CONTENIDOS

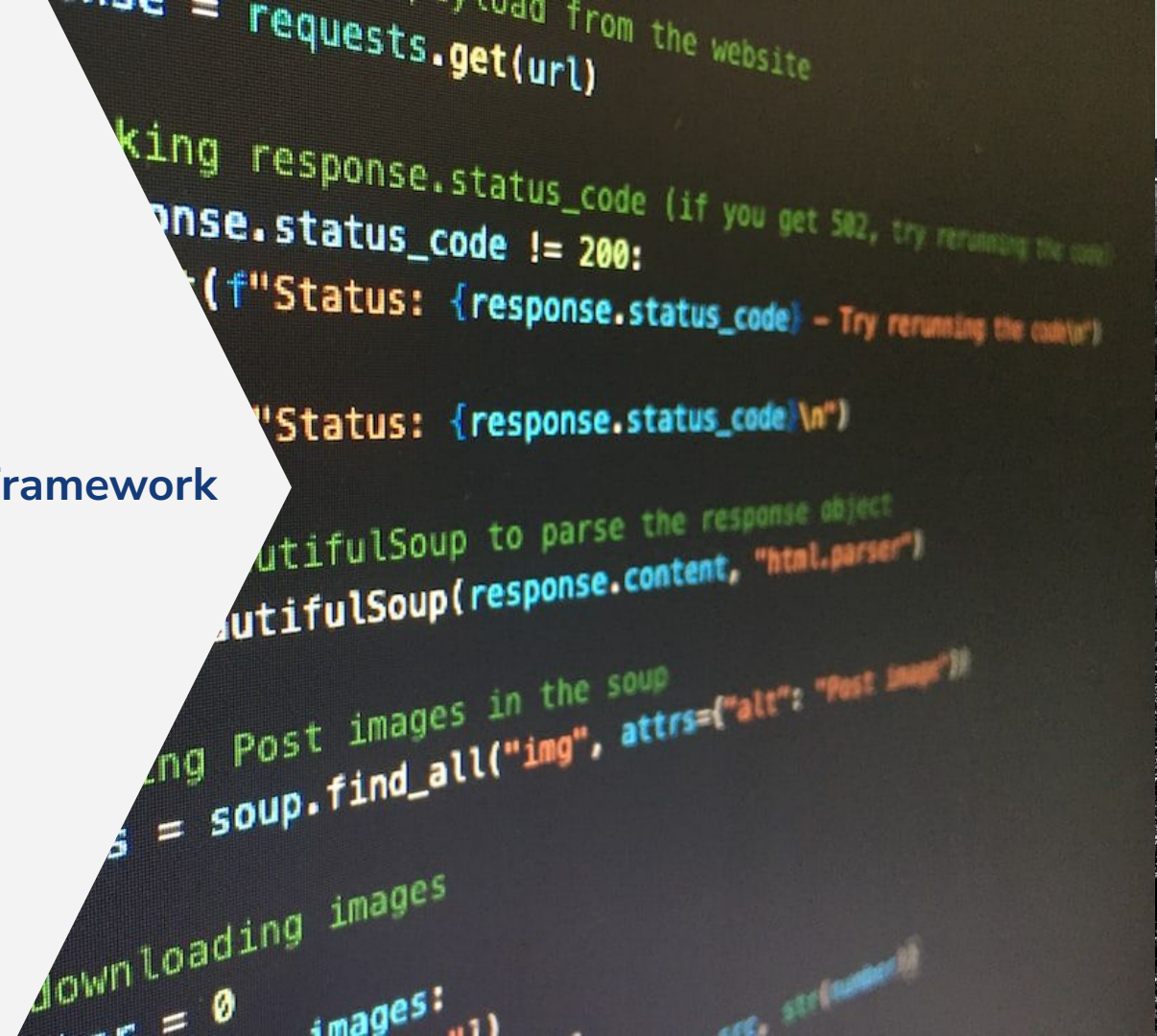
Estructura del Framework

Modelos y Administración

Django Rest Framework

Testing y Logs

Estructura del Framework



¿Que es Django?

Django es uno de los framework gratis y open source más populares para crear aplicaciones web. Es muy veloz, seguro y escalable.

Algunos proyectos que usan Django son:

- Instagram
- Pinterest
- National Geographic

¿Que necesitamos?

Para trabajar con Django necesitamos 3 cosas:

- Navegador
- Consola
- Editor (IDE)

Iniciando proyecto

Mac/Linux

```
virtualenv venv
source venv/bin/activate
pip install django
django-admin startproject ecoapp
cd ecoapp
git init
```

Windows

```
py -m venv venv
.\venv\Scripts\activate
pip install django
django-admin startproject ecoapp
cd ecoapp
git init
```

Explorando estructura

```
(modulo5) ➜  modulo5 tree ecoapp
ecoapp
├── ecoapp
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── manage.py
```

Explorando estructura

primera carpeta ecoapp es solo un contenedor de archivos

manage.py , sirve para trabajar con los comandos disponibles

segunda carpeta interna ecoapp esa es donde se almacenan los archivos de django.

- __init__.py : Nos dice que la carpeta es un paquete.
- asgi.py & wsgi.py : Nos ayudan con el deploy.
- settings.py : Contiene toda la configuración y constantes del proyecto.
- urls.py : Las direcciones para movernos en el proyecto.

https://docs.djangoproject.com/en/6.0/topics/settings/

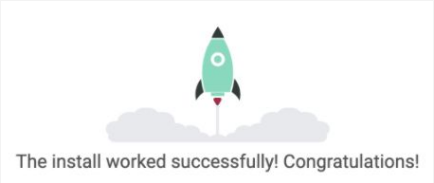
https://docs.djangoproject.com/en/6.0/ref/settings/

Servidor de Desarrollo

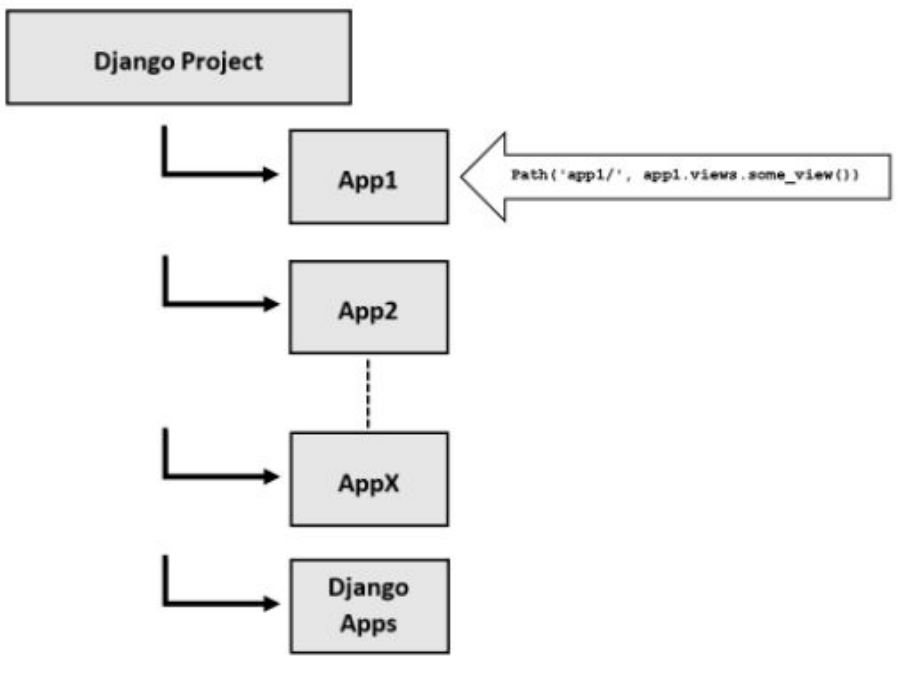
```
python manage.py runserver
```

Descripción de la consola

- **Watching for file …** : A cada cambio que hagas en los archivos, Django lo notará y lo refleja en el servidor.
- **System check identified no issues …** : No hay problemas y se silenciaron 0.
- **You have 18 unapplied migration(s) …** No se ha creado una base de datos efectiva. Date, Version Django
- **Using settings 'ecoapp.settings'** : Tomar el archivo settings-py, ver la variables de configuración y aplicarlas para tenerlas disponibles al crear el código.
- **Starting development … :** Servidor desplegado (iniciado) de manera local.



Proyecto y aplicaciones



- Proyecto : Un proyecto es una colección de configuraciones y aplicaciones para un sitio web en particular. Un proyecto puede contener varias aplicaciones. Una aplicación puede estar en varios proyectos.
- Apps : Una aplicación es una aplicación web que hace algo, por ejemplo, un sistema de blogs, una base de datos de registros públicos o una pequeña aplicación de encuestas.

Creando una Aplicación

```
python manage.py startapp inventario
```

Se creará un nueva carpeta con una estructura de archivos

Hello World

Para este caso realizaremos un “Hola Mundo”

Editando el archivo ***inventario/views.py***

```
from django.shortcuts import render
from django.http import HttpResponse

def index(request):
  return HttpResponse("Hola Mundo")
```

Editando el archivo ***ecoapp/urls.py***

```
from django.urls import path, include


urlpatterns = [
  ...
  path('inventario/', include('inventario.urls'))
]
```

Hello World

Creando el archivo ***inventario/urls.py***

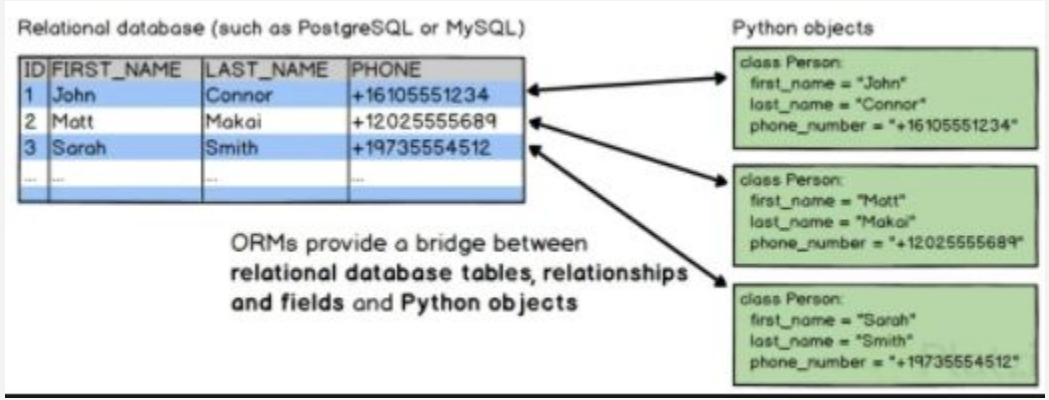
```
from django.urls import path
# from inventario import views
from . import views


urlpatterns = [
  path("", views.index, name="index")
]
```

Modelos

requests.get(url)
response.status_code != 200:
Status: {response.status_code} - Try rerunning the code\n
Status: {response.status_code}\n
BeautifulSoup(response.content, "html.parser")
soup.find_all("img", attrs={"alt": "Post image"})
downloading images

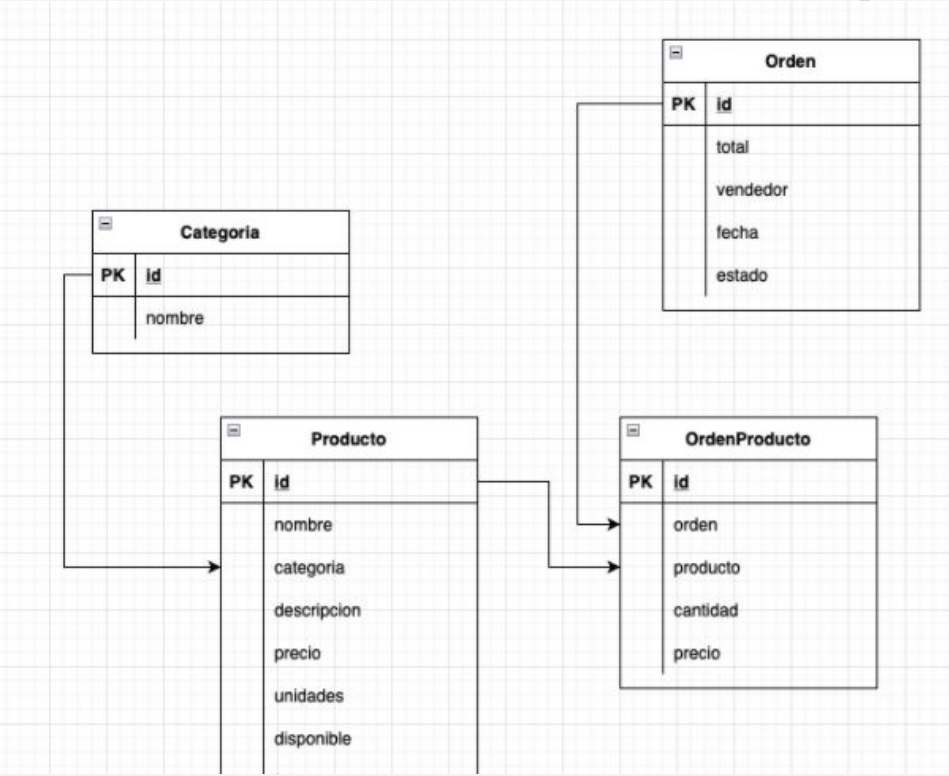
Django ORM

Mapeo Objeto-Relacional o como se conocen comúnmente, **ORM (del inglés Object Relational Mapping)**. Un **ORM** te permite convertir los datos de tus objetos en un formato correcto para poder guardar la información en una base de datos (mapeo) creándose una base de datos virtual donde los datos que se encuentran en nuestra aplicación, quedan vinculados a la base de datos (persistencia).



MODELOS

Modelo E-R; Sistema EcoApp



Instalando una aplicación

settings.py

```
INSTALLED_APPS = [
  ...
  #'inventario.apps.InventarioConfig',
  'inventario',
]
```

Creando un Modelo

https://docs.djangoproject.com/en/6.0/ref/models/fields/

models.py

```python
from django.db import models


class Categoria(models.Model):
    nombre = models.CharField(max_length=100, unique=True)


class ProductUnits(models.TextChoices):
    UNITS = 'u', 'Unidades'
    KG = 'kg', 'Kilogramos'

class Producto(models.Model):
    nombre = models.CharField(max_length=100, unique=True)
    categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE)
    descripcion = models.TextField()
    precio = models.DecimalField(decimal_places=2, max_digits=10)
    unidades = models.CharField(
        max_length=2,
        choices=ProductUnits.choices,
        default=ProductUnits.UNITS
    )
    disponible = models.BooleanField(blank=True, default=True)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
```

Migrando los cambios a la BD

```
python manage.py makemigrations
python manage.py migrate
```

Utilizando la consola interactiva

```
python manage.py shell
```

```
>>> from inventario.models import Categoria
>>> Categoria.objects.all()
>>> q = Categoria(nombre="Hogar")
>>> q.save()
>>> Categoria.objects.all()
```

```
>>> Categoria.objects.all()
<QuerySet [<Categoria: Categoria object (1)>]>
```

El metodo __str__

models.py

```
...
class Categoria(models.Model):
    nombre = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.nombre
...
```

```
>>> from inventario.models import Categoria
>>> Categoria.objects.all()
```

```
>>> Categoria.objects.all()
<QuerySet [<Categoria: Vegetales>]>
```

Filtrando Objetos

```
>>> from inventario.models import Categoria
>>> Categoria.objects.all()
>>> Categoria.objects.get(pk=1)
>>> Categoria.objects.get(nombre="Vegetales")
>>> Categoria.objects.get(pk=2)
>>> Categoria.objects.filter(nombre="Vegetales")
>>> Categoria.objects.get(nombre__contains='eta')
>>> Categoria.objects.filter(nombre__contains='eta')
>>> Categoria.objects.filter(nombre__contains='a', pk=1)
>>> from django.db.models import Q
>>> Categoria.objects.filter(Q(nombre__contains='a') | Q(pk=3))
```

https://docs.djangoproject.com/en/6.0/topics/db/queries/#field-lookups-intro

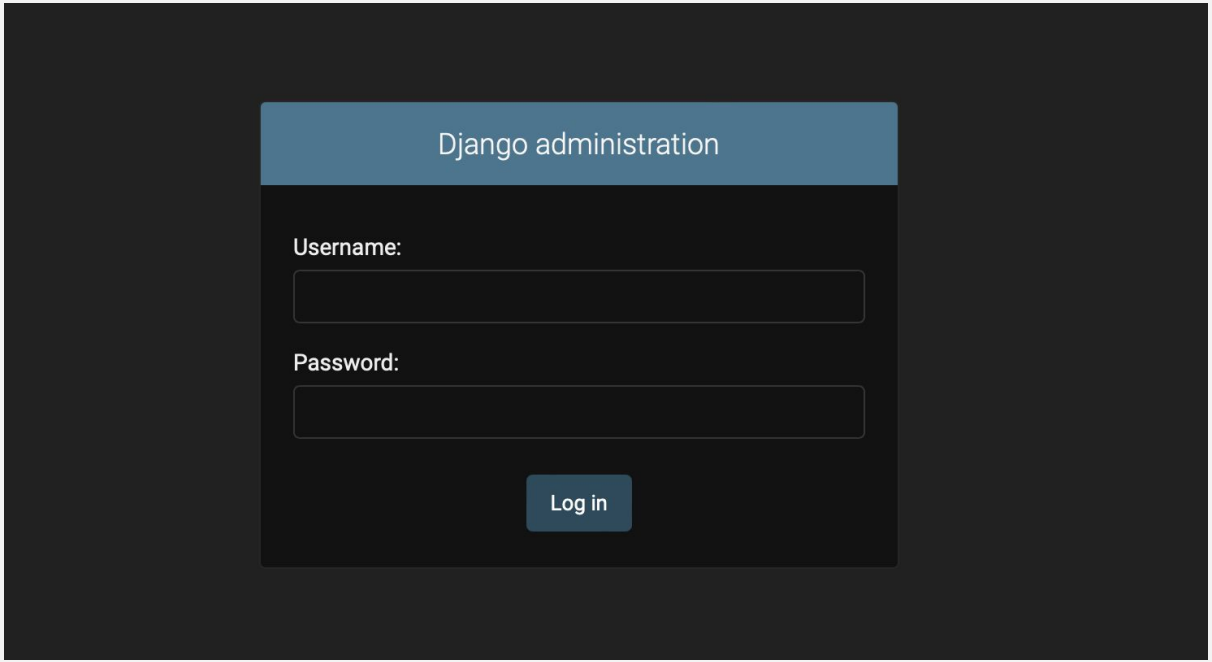
Administrador

Creando el Superusuario

```
python manage.py createsuperuser
```

El Administrador de Django

http://127.0.0.1:8000/admin/



Registrando los modelos en el Administrador

inventario/admin.py

```
from django.contrib import admin
from .models import Categoria
from .models import Producto


admin.site.register(Categoria)
admin.site.register(Producto)
```

Personalizando el Administrador

inventario/admin.py

```python
from django.contrib import admin
from .models import Categoria
from .models import Producto

"""
Categoria
"""
admin.site.register(Categoria)

"""
Producto
"""
class ProductoAdmin(admin.ModelAdmin):
    list_display = ("nombre", "categoria", "precio", "unidades", "disponible",)
    ordering = ["precio"]
    search_fields = ["nombre"]
    list_filter = ("disponible",)

admin.site.register(Producto, ProductoAdmin)
```

https://docs.djangoproject.com/en/6.0/ref/contrib/admin/

Templates

requests.get(url)
response.status_code != 200:
Status: {response.status_code} - Try rerunning the code\n
Status: {response.status_code}\n
soup = BeautifulSoup(response.content, "html.parser")
images = soup.find_all("img", attrs={"alt": "Post image"})
images:

Trabajando con templates

inventario/urls.py

```
from django.urls import path, include
from . import views


urlpatterns = [
    path('/contact/<str:name>', views.contact, name='index'),
    path('', views.index, name='index'),
]
```

Trabajando con templates

inventario/views.py

```python
from django.http.response import HttpResponse
from django.shortcuts import render


def index(request):
    return HttpResponse('Hola Mundo')

def contact(request, name):
    return HttpResponse(f"Bienvenido {name} a la clase de Django")
```

Trabajando con templates HTML

inventario/views.py

```python
from django.http.response import HttpResponse
from django.shortcuts import render

def index(request):
    return HttpResponse('Hola Mundo')

def contact(request, name):
    return HttpResponse(f"Bienvenido {name} a la clase de Django")

def categorias(request):
    return render(request, "categorias.html")
```

Trabajando con templates HTML

inventario/urls.py

```
from django.urls import path, include
from . import views


urlpatterns = [
    path('contact/<str:name>', views.contact, name='contacto'),
    path('categorias', views.categorias, name='categorias'),
    path('', views.index, name='index'),
]
```

Trabajando con templates HTML

inventario/templates/categorias.html

```html
<h1>Categorias</h1>

<ul>
    <li>Categoria 1</li>
</ul>
```

Trabajando con templates, modelos HTML

inventario/templates/categorias.html

```html
<h1>Categorias</h1>

<ul>
    <li>Categoria 1</li>
</ul>
```

Trabajando con templates, modelos HTML

inventario/views.py

```python
from django.http.response import HttpResponse
from django.shortcuts import render
from .models import Categoria

def index(request):
    return HttpResponse('Hola Mundo')

def contact(request, name):
    return HttpResponse(f"Bienvenido {name} a la clase de Django")

def categorias(request):
    categorias = Categoria.objects.all()
    return render(request, "categorias.html", {
        "categorias": categorias
    })
```

Trabajando con templates, modelos HTML

inventario/templates/categorias.html

```
<h1>Categorias</h1>

<ul>
    {% for categoria in categorias %}
    <li>{{categoria}}</li>
    {% endfor %}
</ul>
```

Ejercicio 1

Cree en la vista de categorías un filtro para buscar por el nombre de la categoría:

Ejemplo:

http://127.0.0.1:8000/inventario/categorias?nombre=PALABRA_A_FILTRAR

La siguiente función sirve para capturar query params

```
filtro_nombre = request.GET.get('nombre')
```

GRACIAS